PipedPeer: Figure Preview

Draft for the authors

Evaluation figures from the container sample are stamped “emulated”; the E16 energy figure is a synthetic layout example.

***Abstract*—Every design diagram and evaluation figure planned for the paper, placed at the exact size it will have in the IEEE two-column layout (252 pt single column, 516 pt double column: IEEEtran's own column and text widths), so labels, line weights and spacing can be judged as printed. Captions are drafts. Regenerate with `docs/paper/build-preview.sh`; `docs/paper/FIGURES.md` lists each figure's file, width and paper section.**

Figures

System Design
- Fig. 1: `arch`, double

System Design: Placement
- Fig. 2: `placement`, single
- Fig. 3: `lease`, single

System Design: Environment Reproduction
- Fig. 4: `closure`, single

System Design: Joining Across Networks
- Fig. 5: `nat`, single
- Fig. 6: `trust`, single

System Design: Transparent Interception
- Fig. 7: `shim`, double
- Fig. 8: `shuffle`, single
- Fig. 9: `ddp`, single

System Design: The Cost Model
- Fig. 10: `cost`, single

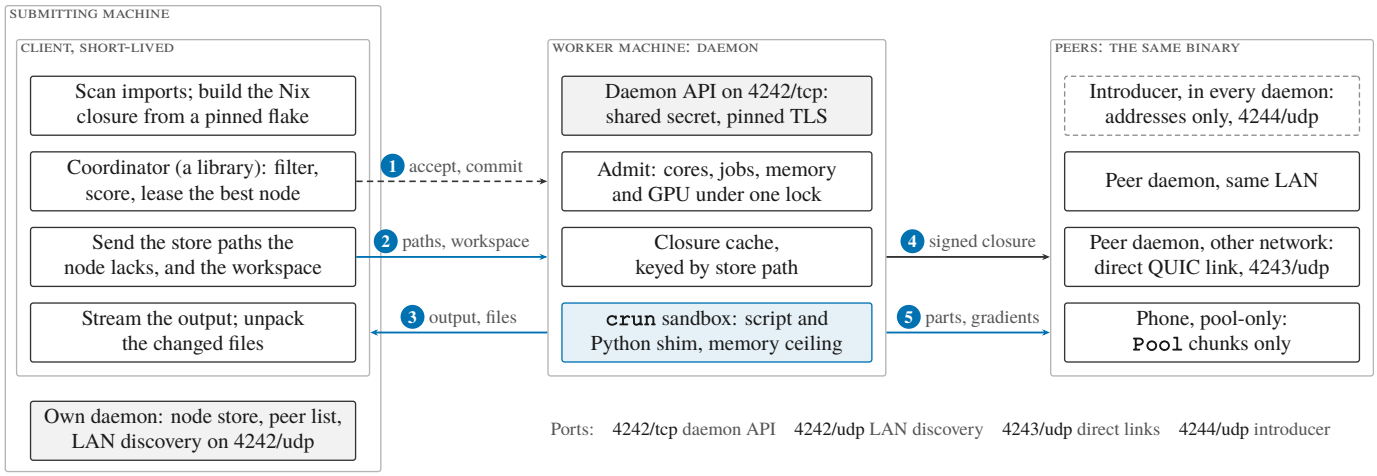
System Design: Implementation
- Fig. 11: `sandbox`, single

Evaluation: Setup
- Fig. 12: `lab`, double
- Fig. 13: `eval-e14-staircase`, single
- Fig. 14: `eval-e17-ready`, single

Evaluation: Microbenchmarks (Q1, Q2)
- Fig. 15: `eval-e7-overhead-cdf`, single
- Fig. 16: `eval-time-breakdown`, single
- Fig. 17: `eval-e17-footprint`, single
- Fig. 18: `eval-e19-ship`, single

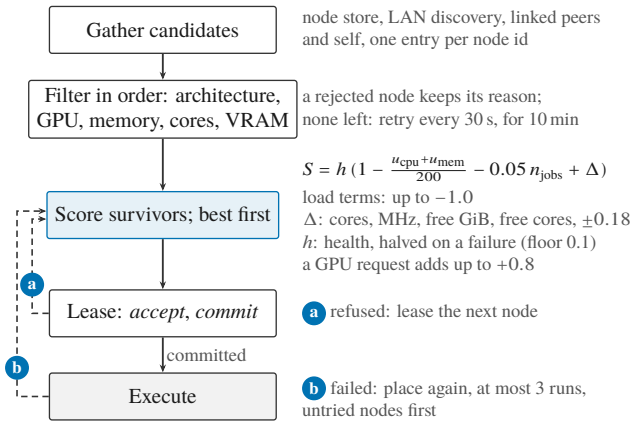
Evaluation: End-to-end runs (Q3)
- Fig. 19: `eval-frontier`, single
- Fig. 20: `eval-strong-scaling`, double
- Fig. 21: `eval-weak-scaling`, single
- Fig. 22: `eval-wall-time`, double
- Fig. 23: `eval-efficiency-heatmap`, single
- Fig. 24: `eval-e4-heterogeneous`, single
- Fig. 25: `eval-e6-kill`, single
- Fig. 26: `eval-cpu-heatmap`, double
- Fig. 27: `eval-swimlanes`, double
- Fig. 28: `eval-net-bytes`, single
- Fig. 29: `eval-offload-share`, single
- Fig. 30: `eval-e12-join`, double
- Fig. 31: `eval-e13-churn`, double
- Fig. 32: `eval-e15-network`, single
- Fig. 33: `eval-e16-energy`, double
- Fig. 34: `eval-e18-straggler`, double
- Fig. 35: `eval-e20-unmodified`, single

Evaluation: Training (Q4)
- Fig. 36: `eval-e8-training`, double



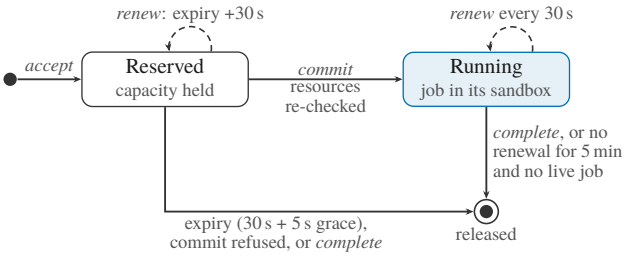
arch: double column, 7.14×2.43 in, System Design

Fig. 1. System design. Both roles are one binary. The client builds the closure, takes a lease (1) and sends only the store paths the worker lacks (2); output and changed files stream back (3). The worker pushes the signed closure to peers (4) and the shim spreads parts and gradients over them (5). Introducers carry addresses, never job data.



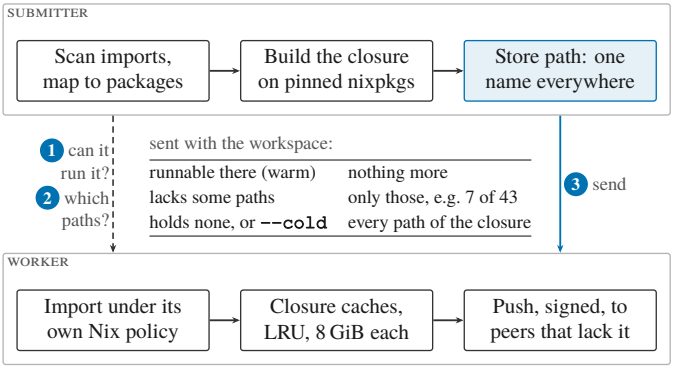
placement: single column, 3.49×2.16 in, System Design: Placement

Fig. 2. Placement in the submitting client. Hard filters run in order; survivors are ranked by the score S of Equation 1. A refused lease moves to the next node (a); a failed run is placed again, at most three times (b).



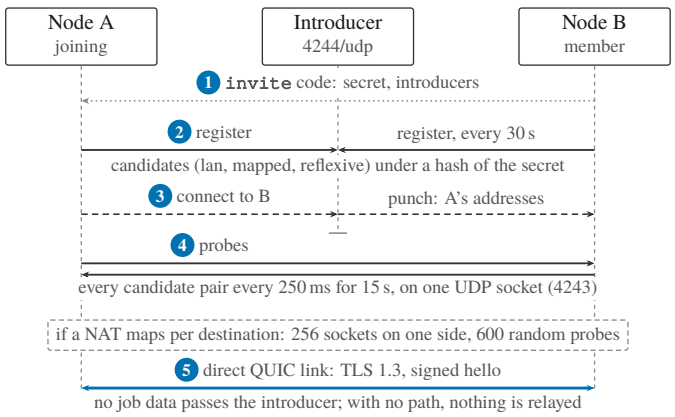
lease: single column, 3.49×1.31 in, System Design: Placement

Fig. 3. Lease states on the worker. A reservation expires 30 s after its last renewal plus a 5 s grace; a running lease is kept while its job lives and reaped only after 5 minutes without renewal.



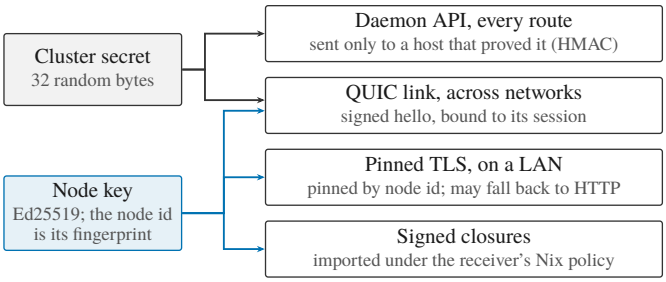
closure: single column, 3.49×1.88 in, System Design: Environment Reproduction

Fig. 4. Environment transfer. The client asks whether the worker can run the closure (1) and which store paths it lacks (2), and sends only those (3). Adding one package sent 7 of 43 paths, 28.6 MB of 206.0 MB.



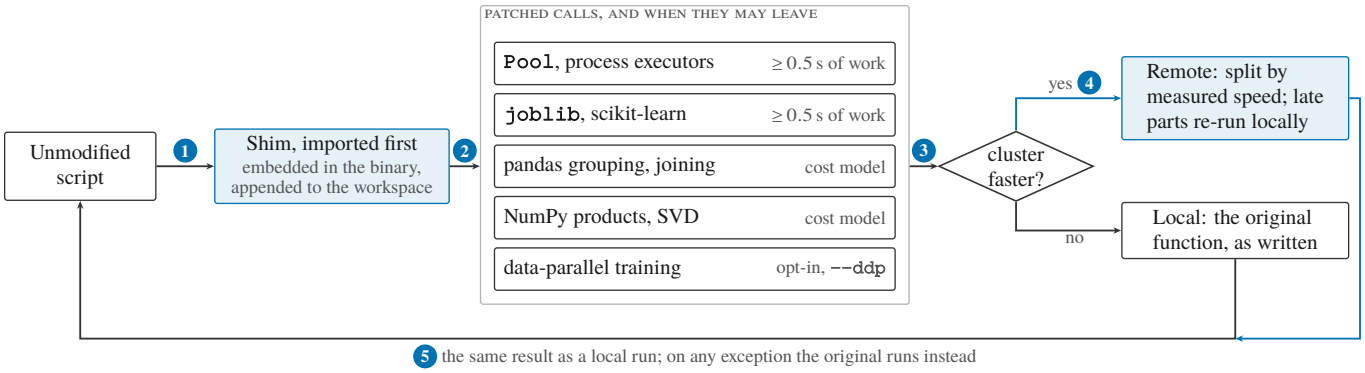
nat: single column, 3.49×2.12 in, System Design: Joining Across Networks

Fig. 5. Joining across networks. Both nodes register their candidates with an introducer (2), which forwards one punch request (3); both sides then probe every candidate pair (4) until a direct QUIC link forms (5). No job data passes through the introducer.



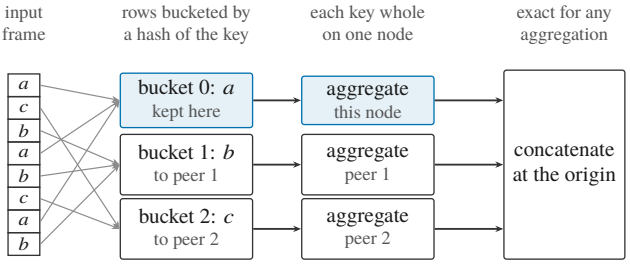
`trust`: single column, 3.49 × 1.42 in, System Design: Joining Across Networks

Fig. 6. Trust. The cluster secret guards every daemon route and the QUIC hello; the node key makes the TLS certificate, signs the hello and signs closures.



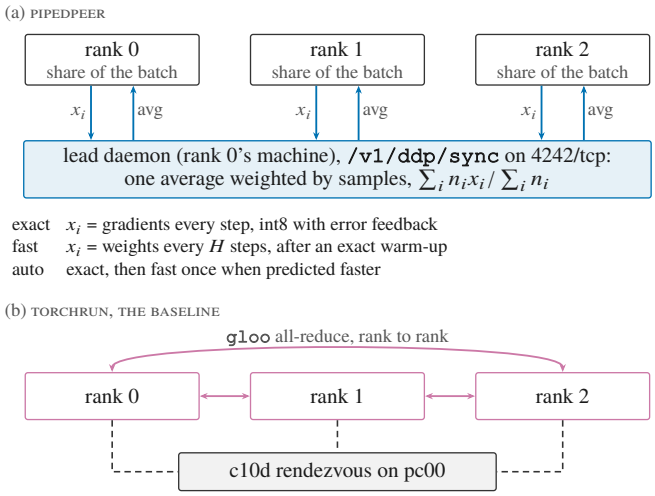
shim: double column, 7.14×1.92 in, System Design: Transparent Interception

Fig. 7. Transparent interception. The shim is imported before the script's first line (1) and patches the listed calls (2). Each call is gated (3); offloaded work is split by measured speed while local cores re-run late parts (4), and the script receives the result a local run would give (5).



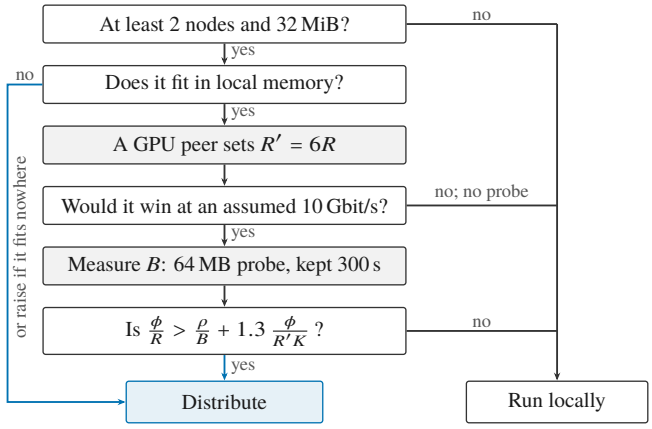
shuffle: single column, 3.49×1.37 in, System Design: Transparent Interception

Fig. 8. Hash shuffle for pandas grouping and joining. Rows are bucketed by a hash of their key, so each key is complete on one node and any aggregation is exact without a combiner; the row order of joins is not preserved.



ddp: single column, 3.49×2.53 in, System Design: Transparent Interception

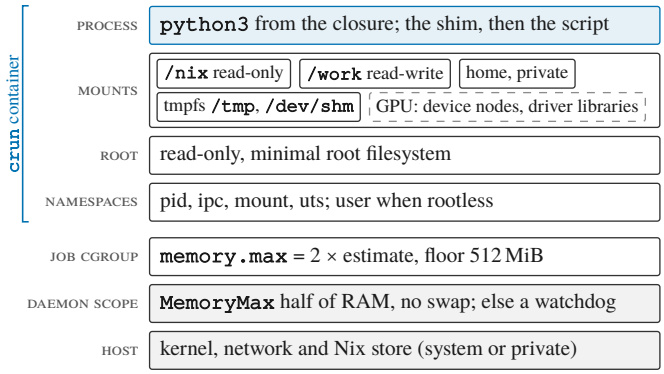
Fig. 9. Data-parallel training. (a) Each PipedPeer rank posts to the lead daemon and receives one sample-weighted average; exact mode sends int8 gradients every step, fast mode weights every H steps. (b) The torchrun baseline meets at a c10d rendezvous and all-reduces between ranks over gloo.



ϕ operations and ρ link crossings per byte; B bandwidth; R, R' local and per-node remote throughput; K nodes holding the environment. Indivisible work (SVD) moves when its transfer costs under half the local time.

cost: single column, 3.49×2.56 in, System Design: The Cost Model

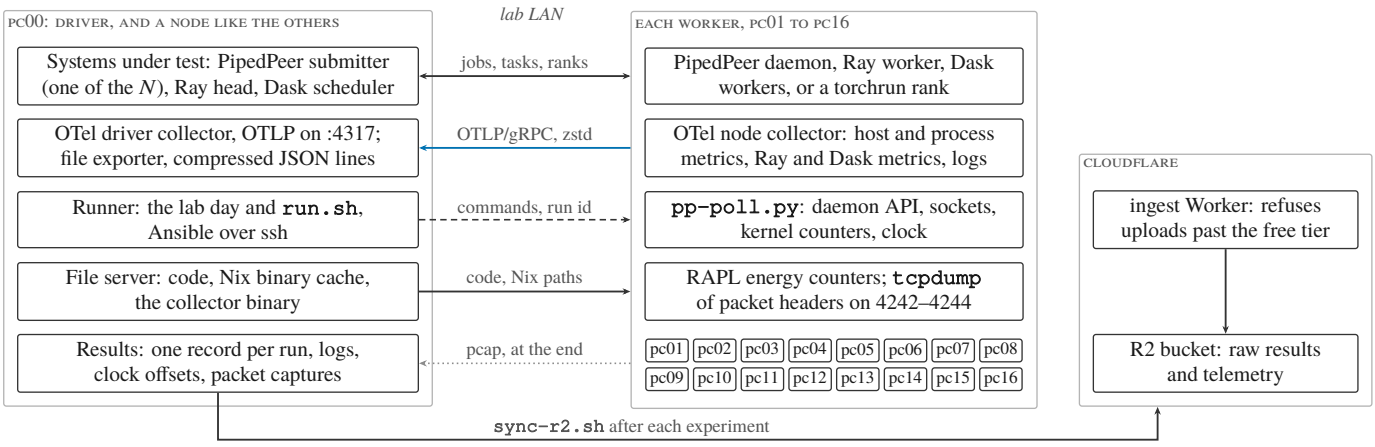
Fig. 10. The cost model as a cascade. Cheap tests run first; bandwidth is measured only when offloading could win at 10 Gbit/s. Work moves when Equation 2 holds or when it does not fit in local memory.



Not provided: a network namespace, a seccomp filter, a capability list. The job runs as uid 0 inside.

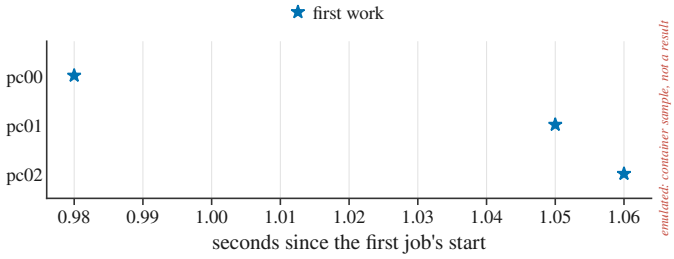
sandbox: single column, 3.49×2.17 in, System Design: Implementation

Fig. 11. The job sandbox. The closure is mounted read-only and the workspace read-write inside a crun container; a per-job cgroup and the daemon's own scope bound memory. The job shares the host network.



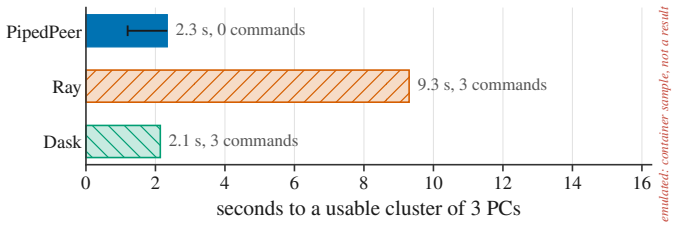
lab: double column, 7.14×2.27 in, Evaluation: Setup

Fig. 12. Lab campaign. pc00 runs the experiments over Ansible, hosts the Ray head, Dask scheduler and PipedPeer submitter, and collects OpenTelemetry from a collector on every PC into files; after each experiment the results go to R2 through an ingest Worker that refuses uploads past the free tier.



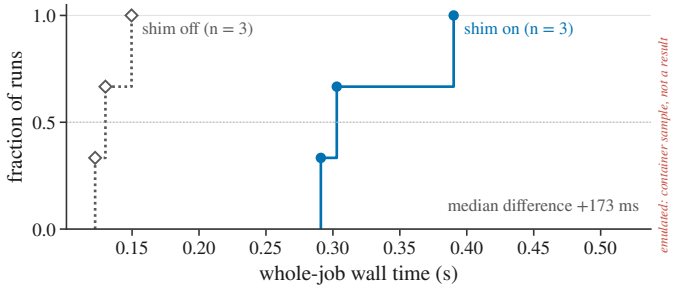
eval-e14-staircase: single column, 3.49×1.35 in, Evaluation: Setup

Fig. 13. Curl to cluster (E14): for every PC, when it printed its LAB-PC line, answered SSH, had PipedPeer installed, joined, and first ran work of the day's first distributed job, relative to the head command on pc00. Dashed lines: cluster ready and first job.



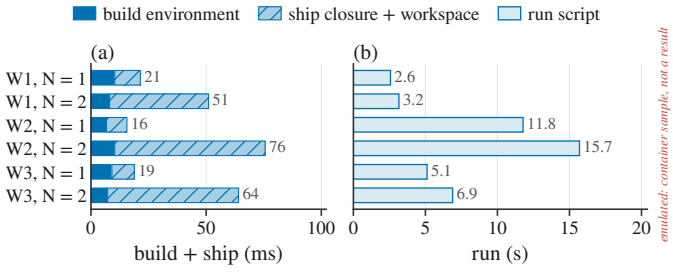
eval-e17-ready: single column, 3.49×1.15 in, Evaluation: Setup

Fig. 14. From nothing running to a cluster that accepts a job (E17): seconds from the first start command until the readiness check passed; the whisker reaches back to the last failed check. The number is the commands a user types after every boot.



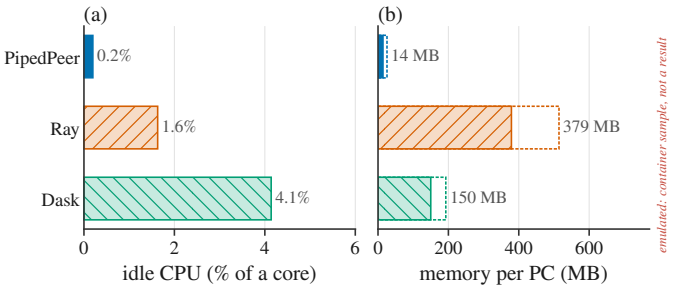
eval-e7-overhead-cdf: single column, 3.49×1.50 in, Evaluation: Microbenchmarks (Q1, Q2)

Fig. 15. Cost of transparent interception (E7): CDF of the whole-job wall time of a script that does no parallel work, with the shim injected and without. The gap between the medians is the per-job overhead.



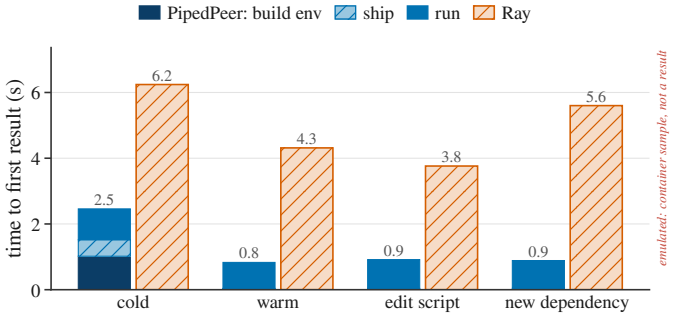
eval-time-breakdown: single column, 3.49×1.40 in, Evaluation: Microbenchmarks (Q1, Q2)

Fig. 16. Where a PipedPeer job's time goes, from its own step records. (a) Building the Nix environment and shipping closure and workspace, in milliseconds; (b) running the script, in seconds. Median over repeats, warm environment cache.



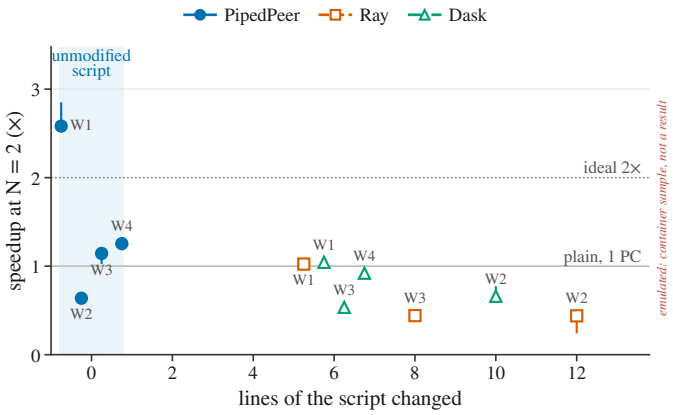
`eval-e17-footprint`: single column, 3.49×1.50 in, Evaluation: Microbenchmarks (Q1, Q2)

Fig. 17. Idle cost of an installed cluster (E17), per PC, averaged over a quiet minute with no job: (a) CPU as a percentage of one core; (b) memory, proportional set size (bar) and resident set size (outline), of every process the system leaves running.



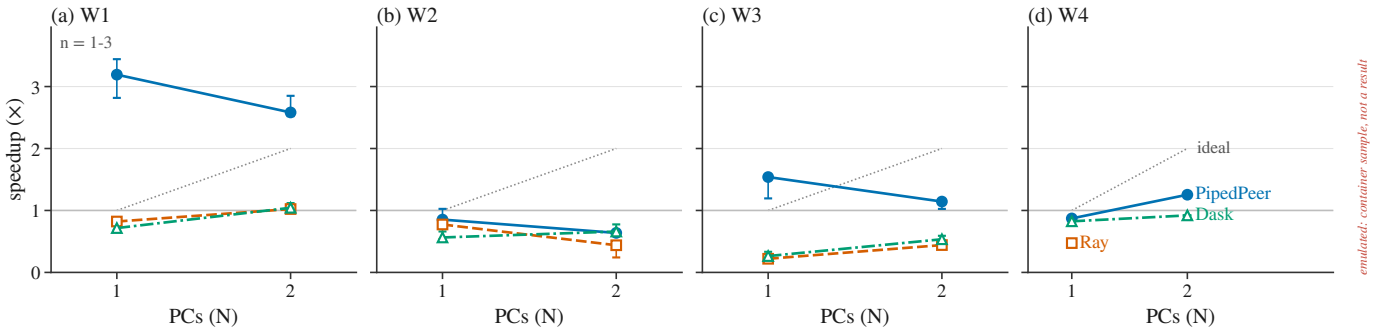
`eval-e19-ship`: single column, 3.49×1.65 in, Evaluation: Microbenchmarks (Q1, Q2)

Fig. 18. Time to first result while iterating on a script (E19), 16 PCs: the first run of a new environment (cold), the same again (warm), one constant edited, and one new dependency. PipedPeer's bar is split into environment build, ship, and the rest of the job. Median over repeats.



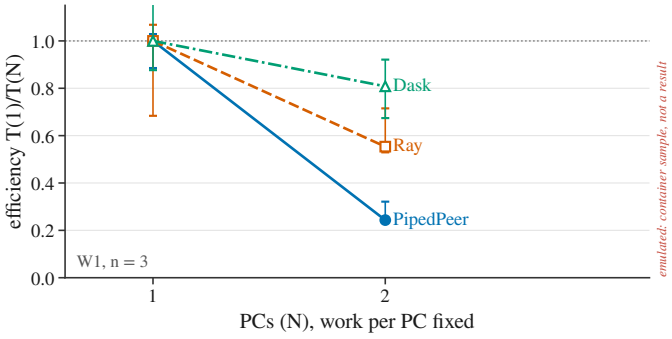
`eval-frontier`: single column, 3.49×2.15 in, Evaluation: End-to-end runs (Q3)

Fig. 19. Porting cost against payoff. Each point is one workload on one system at the largest N; the vertical bar is the interquartile range over repeats. PipedPeer runs the unmodified script (zero lines changed); Ray and Dask need a port.



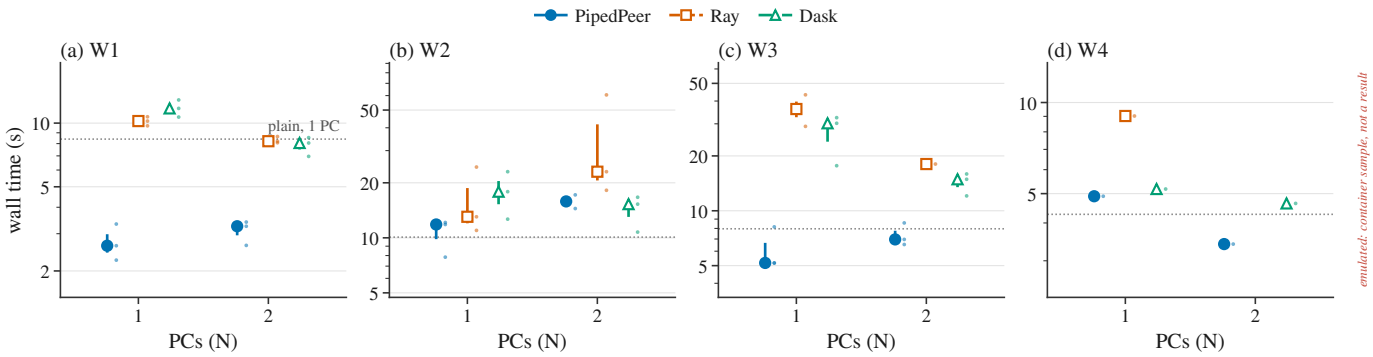
`eval-strong-scaling`: double column, 7.14×1.72 in, Evaluation: End-to-end runs (Q3)

Fig. 20. Strong scaling. Speedup over plain Python on one PC (median; bars are the IQR over repeats) against the number of PCs N. The dotted line is ideal linear speedup; the grey line is parity with plain Python. Panels share the y axis.



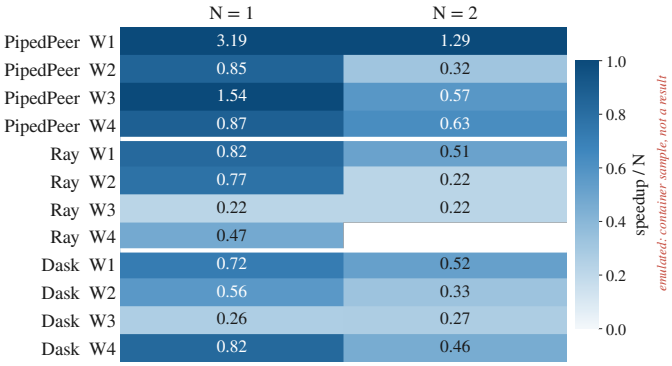
`eval-weak-scaling`: single column, 3.49×1.75 in, Evaluation: End-to-end runs (Q3)

Fig. 21. Weak scaling of W1: the work grows with N, so a perfect system holds efficiency T(1)/T(N) at 1.0 (dotted). Median and IQR over repeats.



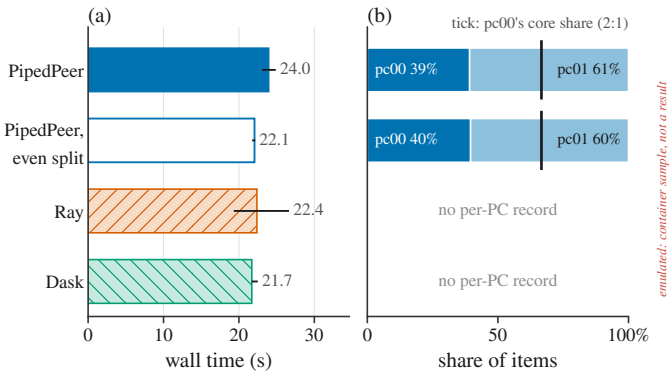
`eval-wall-time`: double column, 7.14×1.85 in, Evaluation: End-to-end runs (Q3)

Fig. 22. Wall time per workload, system and N. The marker is the median, the bar the IQR, and the small dots are the individual repeats; the dotted line is plain Python on one PC. Lower is better; log y axis, each panel with its own range.



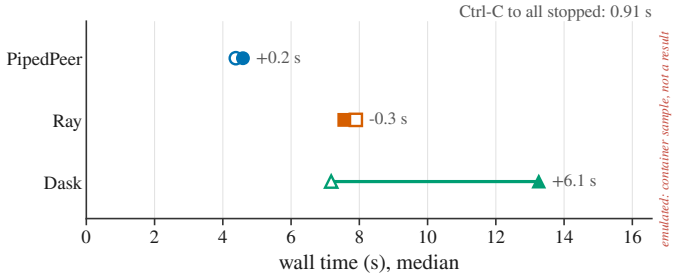
`eval-efficiency-heatmap`: single column, 3.49×1.90 in, Evaluation: End-to-end runs (Q3)

Fig. 23. Parallel efficiency (speedup over plain Python divided by N) for every system, workload and N. 1.00 is linear scaling. Darker is better; values are printed so the figure reads in greyscale.



`eval-e4-heterogeneous`: single column, 3.49×1.95 in, Evaluation: End-to-end runs (Q3)

Fig. 24. Unequal PCs (E4). (a) Wall time, median and IQR. (b) Share of the items each PC computed in one representative run; the black tick is the first PC's share of the cores. The even split is PipedPeer with placement by head count instead of measured speed.



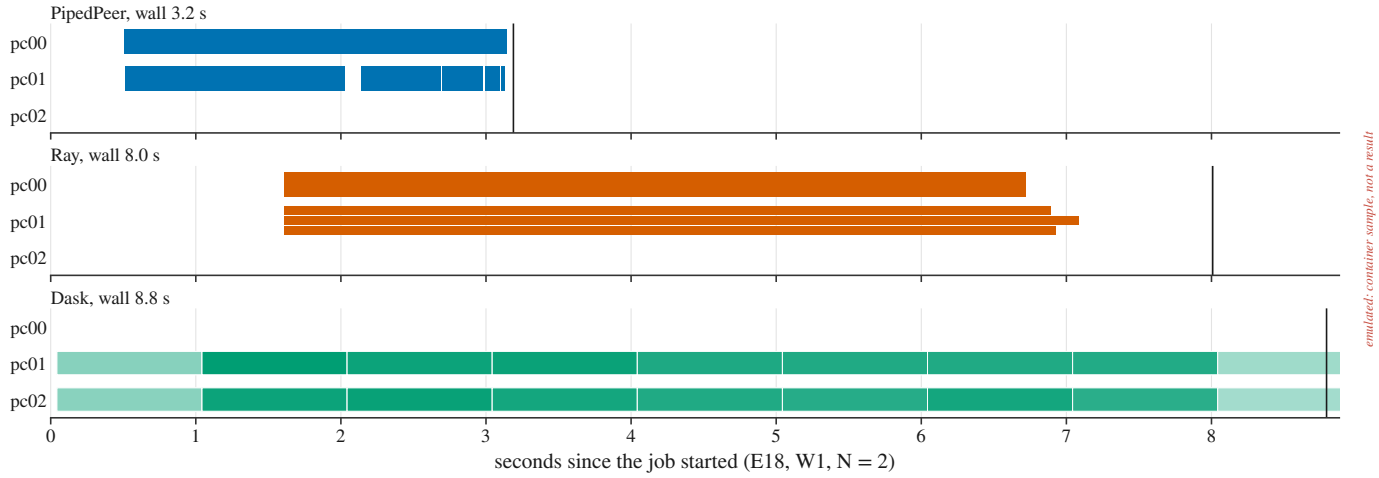
`eval-e6-kill`: single column, 3.49×1.45 in, Evaluation: End-to-end runs (Q3)

Fig. 25. One worker killed with kill -9 mid-run (E6). Hollow: no fault; filled: killed. The label is the added wall time; every run returned the correct result unless marked.



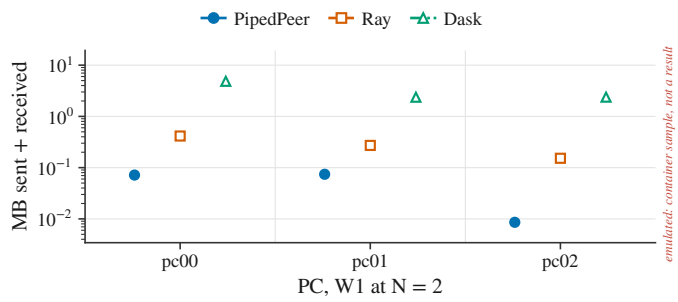
eval-cpu-heatmap: double column, 7.14×2.28 in, Evaluation: End-to-end runs (Q3)

Fig. 26. CPU utilisation of every PC during one W1 run at the largest N (OTel host or cgroup CPU time; busy cores over logical cores, 0.5 s bins). The vertical lines mark the job's start and end; PCs the run did not use are labelled in grey.



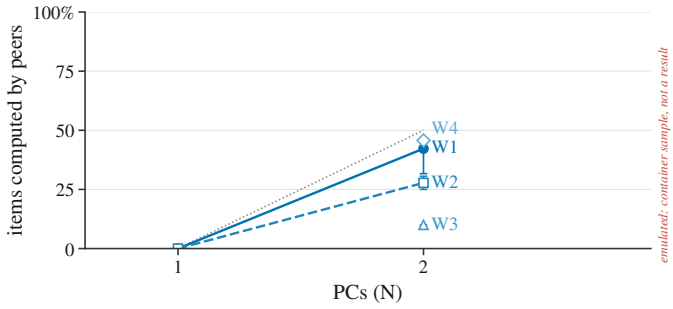
eval-swimlanes: double column, 7.14×2.48 in, Evaluation: End-to-end runs (Q3)

Fig. 27. Where the work ran. Each bar is one chunk of items on the PC that computed it, from each system's own records (PipedPeer's receipt, Ray's timeline, Dask's scheduler). Dask records tasks per second, drawn as one-second cells shaded by count. Rows within a PC are concurrent chunks.



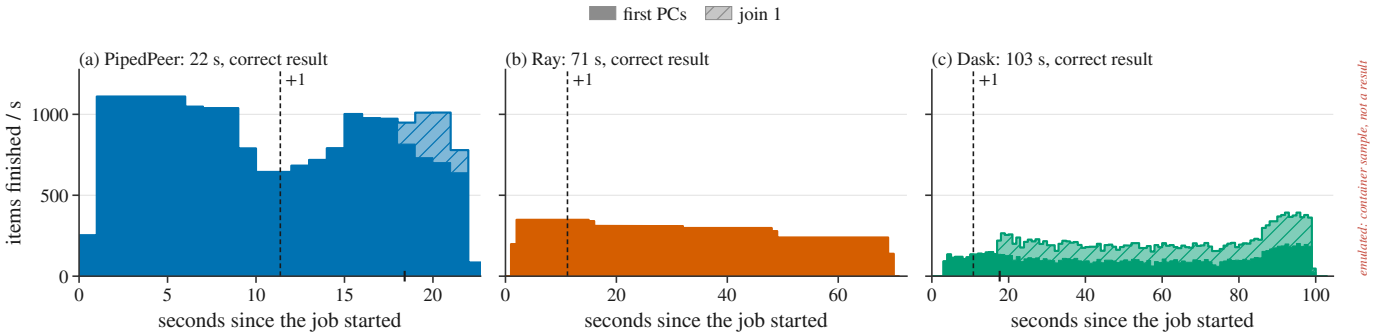
eval-net-bytes: single column, 3.49×1.55 in, Evaluation: End-to-end runs (Q3)

Fig. 28. Network traffic per PC: bytes sent plus received on the lab interface during one W1 run per system at the largest N (OTel system.network.io). Log y axis; pc00 submits the job and hosts the Ray head and the Dask scheduler.



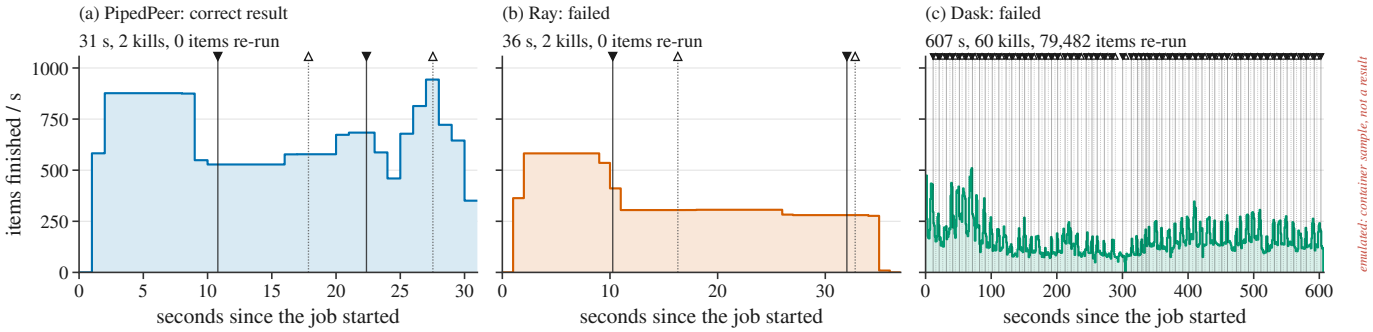
eval-offload-share: single column, 3.49×1.60 in, Evaluation: End-to-end runs (Q3)

Fig. 29. How much work PipedPeer moved off the submitting PC: the share of mapped items computed by peers (from the shim's receipt), median and range over repeats. The dotted line is an even share (N-1)/N.



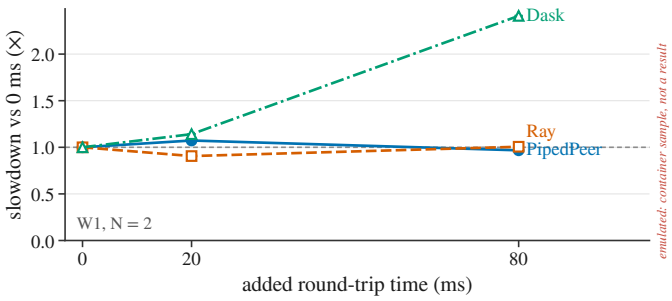
eval-e12-join: double column, 7.14×1.75 in, Evaluation: End-to-end runs (Q3)

Fig. 30. Elastic join (E12). Items finished per second, stacked by cohort: the PCs the job started on, then each group that joined (dashed line, labelled with the PCs added). A cohort that stays at zero is a system that never used the PCs that joined. Panels share the y axis.



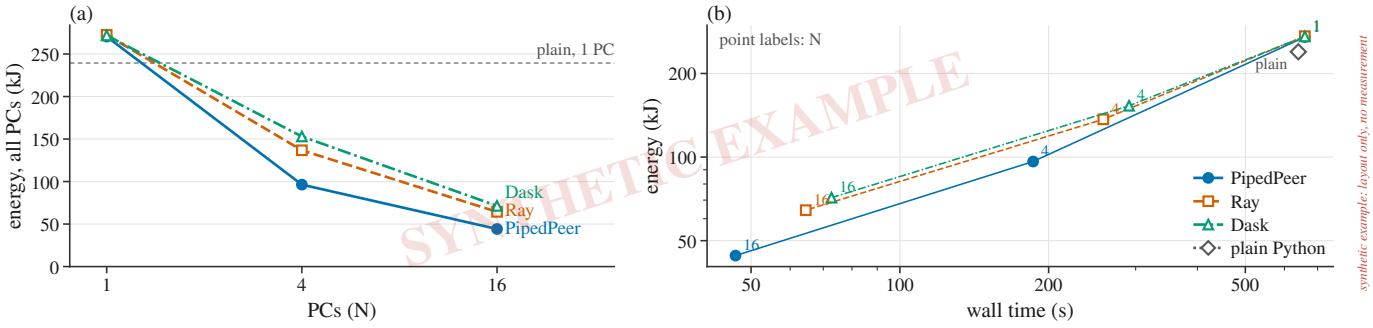
eval-e13-churn: double column, 7.14×1.75 in, Evaluation: End-to-end runs (Q3)

Fig. 31. Churn (E13): a worker is killed with kill -9 periodically (solid line, filled triangle) and restarted a few seconds later (dotted line, hollow triangle). Items finished per second over all PCs; each panel gives the outcome, the wall time against the same job without churn, and the items computed twice.



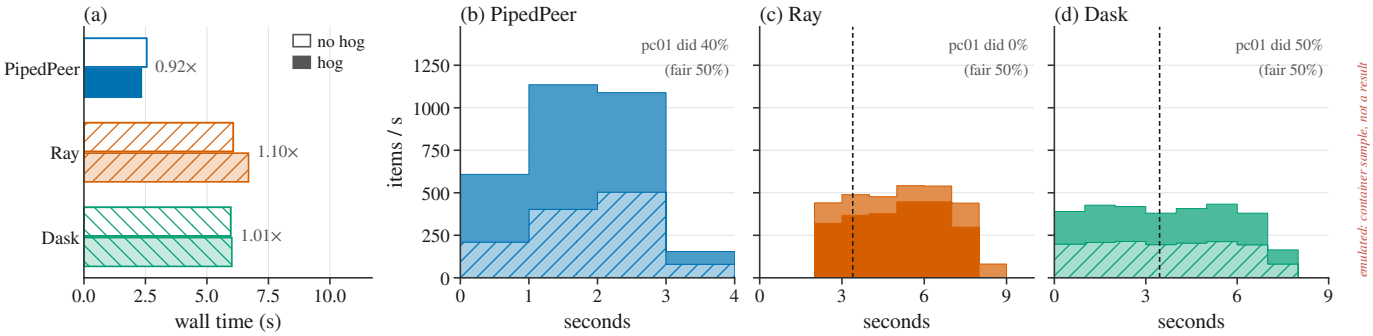
eval-e15-network: single column, 3.49×1.55 in, Evaluation: End-to-end runs (Q3)

Fig. 32. Network sensitivity (E15): W1 wall time with netem delay added between every pair of PCs, relative to the same system with no added delay. Median; bars span the IQR over repeats.



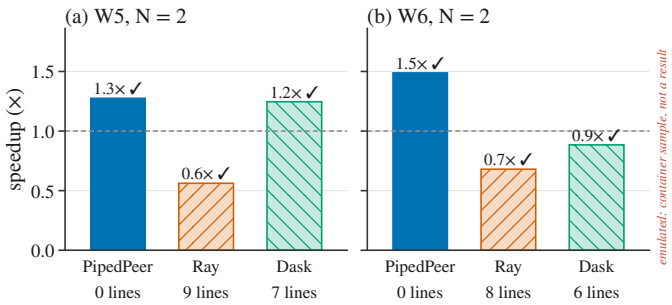
`eval-e16-energy`: double column, 7.14×1.70 in, Evaluation: End-to-end runs (Q3)

Fig. 33. Energy to solution (E16), W1, package energy from RAPL summed over all 17 PCs, idle ones included. (a) Energy against N; the dashed line is plain Python on one PC. (b) Energy against wall time, both log: lower left is faster and cheaper.



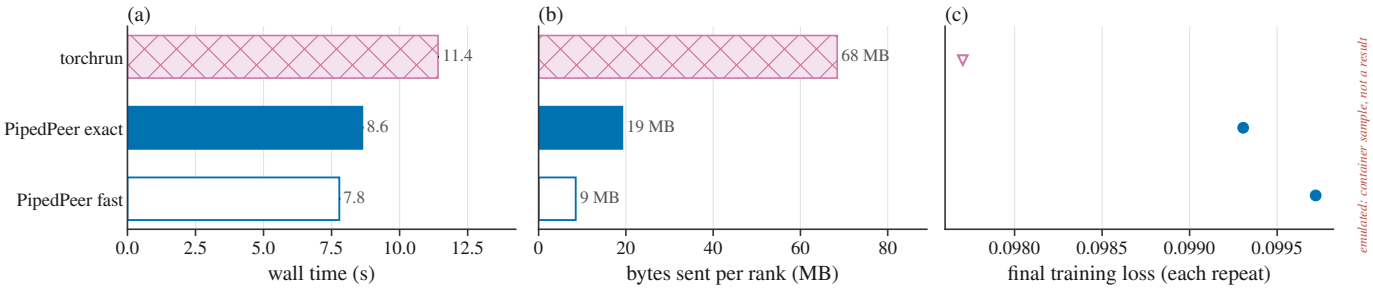
`eval-e18-straggler`: double column, 7.14×1.75 in, Evaluation: End-to-end runs (Q3)

Fig. 34. Noisy neighbour (E18): busy loops on half the cores of one PC, started a few seconds into W1. (a) Wall time without (hollow) and with (filled) the hog; the label is the slowdown. (b-d) Items finished per second in the hog run, stacked per PC, the hogged PC hatched; the dashed line is the hog's start. Panels (b-d) share the y axis.



`eval-e20-unmodified`: single column, 3.49×1.60 in, Evaluation: End-to-end runs (Q3)

Fig. 35. Unmodified pandas and scikit-learn (E20). W5 fits a model per pandas group through joblib; W6 is cross_val_score with n_jobs=-1. Speedup over plain Python on one PC; the tick above each bar marks that every run matched plain Python, and the line under each name is the number of script lines changed.



eval-e8-training: double column, 7.14×1.50 in, Evaluation: Training (Q4)

Fig. 36. Data-parallel training (E8). (a) Wall time, median and IQR. (b) Bytes each rank sent. (c) Final training loss of each repeat. Exact averages gradients every step; fast averages weights every few steps.